PRODUCT OF ARRAY EXCEPT SELF

Implementation in C using Pointers and Dynamic Memory (without Division)

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array nums[]. The array must be passed to a function named product_except_self() using its address, and pointers must be used to pass the array between the calling function and product_except_self().

The function must create an output array answer[] of the same size as nums[], such that answer[i] holds the product of all the elements of nums[] except nums[i]. The elements of answer[] must be displayed in their original order, separated by commas. Division must not be used, and it may be assumed that the product of every prefix and suffix of nums[] fits within a 32-bit integer.

Examples

Example	Input Array	Output
1	`1 2 3 4`	`24, 12, 8, 6`
2	`-1 1 0 -3 3`	`0, 0, 9, 0, 0`

2. Objectives

- To read an array of user-defined size using dynamic memory allocation (malloc).
- To pass and return arrays between functions using pointers.
- To access array elements using pointer notation, *(ptr + i).
- To solve the problem in linear time without using the division operator.

3. Concepts Used

- **Pointers:** nums and answer are accessed through pointers and passed between functions by address.
- **Dynamic memory allocation:** malloc() creates arrays whose size is known only at run time, and free() releases them.
- **Functions:** product_except_self() takes int *nums and int n, and returns int * pointing to answer[].
- **Loops:** for loops traverse the array left-to-right and right-to-left.
- **Prefix and suffix products:** the core idea that removes the need for division.

4. Algorithm / Approach

A simple solution would multiply all elements and divide by nums[i], but division is not allowed (and fails when an element is zero). Instead, the product of all elements except nums[i] is split into two parts:

answer[i] = (product of elements to the left of i) × (product of elements to the right of i)

Because neither part contains nums[i], the element itself is automatically excluded. Both parts are computed with a single running variable each, so no extra arrays are needed.

Steps

1. Read the size n and allocate memory for nums[] using malloc().
2. Read the n elements into nums[] using pointer notation (nums + i).
3. Call product_except_self(nums, n), passing the base address of nums[].
4. Inside the function, allocate answer[] of size n and set prefix = 1, suffix = 1.
5. Left-to-right pass: for each i, store answer[i] = prefix, then update prefix = prefix × nums[i].
6. Right-to-left pass: for each i, update answer[i] = answer[i] × suffix, then suffix = suffix × nums[i].
7. Display answer[] in the original order separated by commas, and return its address to main().
8. main() releases both arrays using free().

Dry Run (Example 1: nums = 1 2 3 4)

Index i	nums[i]	Left Product	Right Product	answer[i]
0	1	1	2 × 3 × 4 = 24	24
1	2	1	3 × 4 = 12	12
2	3	1 × 2 = 2	4	8
3	4	1 × 2 × 3 = 6	1	6

Hence the output is 24, 12, 8, 6, which matches the expected result.

5. Source Code

File name: product_except_self.c

```
/*
 * Program : Product of Array Except Self
 * Author  : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 *   Reads a variable-length integer array nums[] and, without using
 *   division, builds answer[] where answer[i] is the product of all
 *   elements of nums[] except nums[i]. Arrays are passed between
 *   main() and product_except_self() using pointers.
 */

#include <stdio.h>
#include <stdlib.h>

/* Builds, displays and returns answer[]; caller must free() it. */
int *product_except_self(int *nums, int n)
{
    int *answer = (int *)malloc(n * sizeof(int));
    int prefix = 1, suffix = 1;
    int i;

    if (answer == NULL)
        return NULL;

    /* Pass 1: answer[i] = product of all elements to the LEFT of i */
    for (i = 0; i < n; i++)
    {
        *(answer + i) = prefix;
        prefix *= *(nums + i);
    }

    /* Pass 2: multiply by product of all elements to the RIGHT of i */
    for (i = n - 1; i >= 0; i--)
    {
        *(answer + i) *= suffix;
        suffix *= *(nums + i);
    }

    /* Display answer[] separated by commas */
    printf("Output: ");
    for (i = 0; i < n; i++)
    {
        printf("%d", *(answer + i));
        if (i < n - 1)
            printf(", ");
    }
    printf("\n");

    return answer;
}

int main(void)
{
```

```
    int n, i;
    int *nums, *answer;

    printf("Enter the size of the array: ");
    if (scanf("%d", &n) != 1 || n < 1)
    {
        printf("Invalid size.\n");
        return 1;
    }

    nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", nums + i);

    answer = product_except_self(nums, n);   /* pass address of nums[] */
    if (answer == NULL)
    {
        printf("Memory allocation failed.\n");
        free(nums);
        return 1;
    }

    free(nums);
    free(answer);
    return 0;
}
```

6. Sample Output

Example 1

```
Enter the size of the array: 4
Enter 4 elements: 1 2 3 4
Output: 24, 12, 8, 6
```

Example 2

```
Enter the size of the array: 5
Enter 5 elements: -1 1 0 -3 3
Output: 0, 0, 9, 0, 0
```

7. Test Cases

No.	Input Array	Output	Remark
1	1 2 3 4	24, 12, 8, 6	All positive numbers

No.	Input Array	Output	Remark
2	`-1 1 0 -3 3`	`0, 0, 9, 0, 0`	Negative numbers and one zero
3	`0 0 2 3 4`	`0, 0, 0, 0, 0`	Two zeros
4	`5 -4`	`-4, 5`	Minimum useful size (2)
5	`7`	`1`	Single element (empty product = 1)

All test cases were compiled with gcc -Wall -Wextra and executed successfully with no warnings or errors.

8. Complexity Analysis

- **Time complexity:** O(n) – the array is traversed twice (left-to-right and right-to-left).
- **Space complexity:** O(1) extra space apart from the output array answer[], since only the variables prefix and suffix are used.

9. Conclusion

The program correctly computes the product of all array elements except the element at each position without using division. The prefix and suffix product technique solves the problem in a single pass each way, handles zeros and negative numbers correctly, and works for any array size through dynamic memory allocation. The task also demonstrated how pointers can be used to pass arrays between functions and how heap memory is allocated and freed safely.